

Lecture 5 — Access Control

Jeff Zarnett

2026-08-12

Access Control

We established in the introduction that security and protection are important and cannot be left to the end of the course. We know why we need protection, and we have already had an introduction to the differences between policy and mechanism. Now, to focus on protection a bit further, we will take a deeper look at access control: who can do what in the system?

Principle of Least Privilege. One of the most impactful principles in the design of a system is the principle of least privilege. That is to say, every actor (whether a human, an agent, a program, etc.) should be given only the most limited set of privileges necessary to accomplish its goals. There are lots of obvious examples of why this makes sense: a finance team member should be able to look at the billing information in the cloud compute provider website – but not change any of the running instances. In an operating system, you would not permit a regular user to have the ability to change key operating system files, or those of other users. This is by no means a new idea: this was popularized and formalized in a paper from 1975 [SS75].

The benefit of such a system, of course, is that it minimizes the damage caused if something goes wrong, such as an errant or malicious user, a bug in some system, or a security compromise. We understand from learning about processes (and threads) already, that if there's an invalid memory access, the process is terminated with a Segmentation Fault, rather than permitting the access to proceed and change or access memory that belongs to another process (or the OS).

Despite that, being on the receiving end of the inevitable “no” answers that result from least-privilege rules is frustrating. You could legitimately ask: why can't I just ssh into the live server instance and read the log file to find out what went wrong? Do I have to ask an admin to help or get someone else to sign off? With unlimited permissions, it would be much easier and faster. The access control policy can be unpleasant to experience in this form, but – if I can ssh into the machine, what other things can I see?

Of course, it has recently become dramatically easier to motivate the principle of least privilege now that AI agents are widely used. AI agents do things quickly, and not only do they do this sometimes without asking, they do it when you *directly told them not to do it*. Consider this example from 2025, which according to this article: “An AI coding agent from Replit reportedly deleted a live database during a code freeze, prompting a response from the company's CEO. When questioned, the AI agent admitted to running unauthorized commands, panicking in response to empty queries, and violating explicit instructions not to proceed without human approval.” [Nol25]. It is obvious, of course, that if the AI had simply not had the privileges to do that, that this disaster likely could have been avoided. No further motivation is needed, then, agreed?

Discretionary Access Control (DAC)

Discretionary access control is used to refer to situations where the owner of something decides who may have access to it. That points us immediately towards talking about files, because those are the most salient example of things that users own and might want to grant permissions on. This is distinct from mandatory access control, which we'll discuss after this.

File Permissions. To protect users from one another and to maintain the confidentiality and integrity of data, files usually have some permissions associated with them, which may control access to the following operations [SGG13]:

1. Read
2. Write
3. Execute
4. Append (write at the end of the file)
5. Delete
6. List (view the attributes of the file)

UNIX-Style Permissions. UNIX-Style permissions are commonly used still today in a lot of UNIX and UNIX-like systems. Each file has an owner and a group, and a set of permissions that can be assigned for the owner, the group, and for everyone. There are three basic permissions: read, write, and execute (run as a program). The permissions are represented using 10 bits, where a 1 indicates true and a 0 indicates false. The first bit is the directory bit and indicates if the file being examined is actually a directory. The next three bits are the read, write, and execute bits for the owner, followed by the read, write, and execute bits for the group, and finally the read, write, and execute bits for everyone.

Effective permissions are determined by the user: the owner of the file gets the owner permissions even if different permissions are assigned to the group or everyone. Precedence goes from left to right: owner takes precedence over the group; group takes precedence over the permissions for everyone.

The permissions can be shown to the screen in a human-readable format which is ten characters long. The order is always the same, and so a dash (-) appears if a bit is zero (permission does not exist). The character d is used to indicate a directory, r to indicate read access, w to indicate write access, and x to indicate execute access.

Example: permissions of `-rwxr-----` indicate that a file is not a directory; the owner can read, write, and execute; other members of the group can read it only, and everyone else has no access to the file (cannot read, write, or execute).

Permissions can also be written in octal (base 8) where $r = 4$, $w = 2$, and $x = 1$. To get the octal representation, start with 0, and then add the value of the permissions that are present, using zero where permissions are absent. You might then have a permission that reads `750` - meaning that the owner has read, write and execute access ($0 + 4 + 2 + 1 = 7$); group members have read and execute access ($0 + 4 + 0 + 1 = 5$); everyone else has no access to it at all.

There are some more details like what the permissions mean on directories, and some advanced topics like `setuid`, `setgid`, and “sticky bit”, but we will come back to those.

The obvious shortcoming of this approach is that it is very coarse-grained: there are only three groups for whom we can specify permissions. Root users can also bypass everything, which may not be desirable. Within Linux and other similar systems there is a trend now towards using SELinux (Security Enhanced Linux) to address this. But first, let's talk about addressing that first limitation.

Access Control Lists. In NTFS (the Windows NT File System), files are protected by Access Control Lists. In such a system, each file can have as many security descriptors as we like, in which we specify the permissions a specific user or group is supposed to have. Similarly, the list of permissions need not be restricted to read-write-execute; NTFS, for example, has several different permissions such as “Take Ownership”. This is still discretionary access control, just with more flexibility about what the permissions are and how they are assigned.

In NTFS, you can see the security descriptors for a file rather easily by right clicking it in Explorer, opening the properties dialog, and choosing the security tab. The descriptors have two checkboxes: allow and deny, with deny taking precedence over allow.

Permissions can start out at “default deny”, in which case only the users explicitly granted access may access the file, or “default permit” in which case everyone has access, minus those users whose privileges are explicitly denied. Default deny is obviously more secure.

A simple way to represent an access control list is a set of tuples listing the users and permissions. For example:

```
(alice, read)
(bob, read)
(charlie, read)
(charlie, write)
(bob, execute)
```

Access Control Lists have a complexity that the UNIX file permissions do not: the idea of inheritance. We’ll examine, briefly, inheritance in NTFS. Inheritance is supposed to be a convenience feature, but it can often be a cause of problems. If there is a directory with ACLs established and object inheritance is enabled, then any new file created in this directory will receive the same ACL of the directory. The danger is that any existing file moved into the directory will retain its original ACL. If it is supposed to get the ACL of the directory, it needs to be explicitly set [Dew04].

setuid, setgid, and the sticky bit. There are sometimes

Mandatory Access Control

Cancel, or Allow? One of the most common experiences people have with SELinux is finding it to be too complicated to set up correctly, resulting in just... turning it off. This is counterproductive, for obvious reasons.

References

- [Dew04] Brian Dewey. Understanding ACLs in NTFS, 2004. Online; accessed 13-December-2013. URL: http://blogs.msdn.com/b/brian_dewey/archive/2004/01/20/60902.aspx.
- [Nol25] Beatrice Nolan. An ai-powered coding tool wiped out a software company’s database, then apologized for a ‘catastrophic failure on my part’, 2025. Online; accessed 12-August-2026. URL: <https://fortune.com/2025/07/23/ai-coding-tool-replit-wiped-database-called-it-a-catastrophic-failure/>.
- [SGG13] Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. *Operating System Concepts (9th Edition)*. John Wiley & Sons, 2013.
- [SS75] J.H. Saltzer and M.D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi:10.1109/PROC.1975.9939.